

Reef Riot - Web App Blueprint

Python + React website MVP for a 24-hour hackathon

Version 1.0 | Browser build blueprint

Product type	One-page website with one interactive play scene
Frontend stack	React + Vite + Tailwind + Framer Motion
Backend stack	Python FastAPI for lightweight API endpoints and scoring/facts
Delivery standard	Stable local demo with polished transformation and short pitch

Blueprint optimized for a 24-hour hackathon and a browser-first demo.

1. Product Definition

Reef Riot becomes a single-page web experience with one playable interactive reef scene at its center. The site opens with a cinematic landing screen, moves into a short mission briefing, lets the user control a rogue shark through one compact corruption-clearing loop, and ends with a visible reef restoration plus a short environmental takeaway.

North star

- The demo must be understandable in under 20 seconds with minimal explanation.
- The interactive reef scene is the hero feature; everything else supports it.
- Ship one polished sequence, not a large unfinished website.

Item	Definition
Website type	Narrative landing page plus one browser-playable interaction zone
Core message	Human-built extraction and waste systems damage reefs; restoration becomes visible when those systems are removed
Primary user action	Move through the reef, clear 3 corruption nodes, watch the environment heal
Success condition	A judge can see the before/after contrast and understand the environmental theme instantly
Hard scope line	No accounts, no database-heavy features, no multiplayer, no large map, no complex game engine in browser

2. Page Flow and User Journey

Section	What the user sees	Primary action	Outcome
Landing hero	Glitchy underwater title screen with corrupted reef mood	Click Dive In	Smooth transition into mission
Mission briefing	Short story panel with controls and objective list	Read for 10 seconds, press Start	User understands role and goal
Playable reef scene	One compact interactive reef arena in the	Move shark and clear 3 nodes	Restoration meter rises; reef updates

Section	What the user sees	Primary action	Outcome
	browser		
Transformation moment	Large visual recovery pulse and environment shift	Watch finale	Project delivers emotional payoff
Impact section	Short facts and project takeaway	Read / replay	Judges leave with the theme, not just visuals

3. Visual and UX Blueprint

The website should feel like a cyber-ocean poster that becomes a living reef. The corrupted state uses dark navy, toxic green, glitch magenta, dead coral gray, scanline/noise overlays, and machine silhouettes. The restored state shifts to bright aqua, seafoam, coral pink, warm orange, and clean bioluminescent glow.

- **Landing section:** full-screen, large REEF RIOT title, shark silhouette, system-warning text, one strong CTA.
- **Mission section:** compact overlay card with short copy, no long lore paragraphs.
- **Playable scene:** full-width, viewport-sized section with layered 2D assets or a lightweight canvas scene.
- **Ending section:** bright reef background, three short environmental facts, replay button, team branding.

UI rule

- Keep the interface sparse: restoration meter, objective list, one context prompt, one ending panel.
- Every completed action must create visible change in the scene immediately.

4. Playable Scene Design

The playable section is a single compact reef arena. The shark starts in the center. Three visually distinct pollution targets sit around the arena so the user always knows where to go.

Suggested arena logic

[Node C: Extraction Drill]
tallest machine / red core

[Node A: Plastic Compactor] [Spawn] [Node B: Ghost Net]
trash cloud / debris trapped creature / net

small arena boundary = reef wall / rocks / fog edge

Node	Theme	Interaction	Visual payoff
Plastic Compactor	Plastic waste and trash dumping	Click or ram 3 times	Trash storm stops; water clears slightly
Ghost Net	Abandoned gear trapping life	Hold / drag / click to cut free	Creature escapes; coral gains color
Extraction Drill	Industrial extraction and reef damage	Repeated hit or charged shutdown	Big pulse triggers final restoration

5. Frontend Architecture - React

Use React with Vite for speed. The website should be structured as one page with clear sections and a dedicated interactive scene component. The interactive portion can be built in plain React with layered divs/SVGs or with a lightweight canvas if one teammate is comfortable with it.

```
src/
  App.jsx
  pages/
    LandingSection.jsx
    MissionSection.jsx
    PlaySection.jsx
    ImpactSection.jsx
  components/
    SharkAvatar.jsx
    PollutionNode.jsx
    RestorationMeter.jsx
    ObjectiveList.jsx
    GlitchOverlay.jsx
    FactCard.jsx
  assets/
    shark/
    reef/
    nodes/
    ui/
  lib/
    api.js
    constants.js
```

Frontend responsibility	Implementation note
App shell	Own smooth scrolling, section transitions, and overall layout
State	Track node completion, progress percentage,

	current reef state, and end panel
Animation	Use Framer Motion for hero transitions, glow pulses, fades, and UI entrances
Styling	Use Tailwind for speed and consistent spacing
Interaction layer	Keep controls simple: pointer-follow, click, drag, or keyboard if already comfortable
Performance goal	Fast load and smooth demo on a laptop browser

6. Backend Architecture - Python

The backend should stay intentionally light. Use Python with FastAPI only for the pieces that benefit from separation, such as providing fact content, tracking a small score or session summary, or returning node descriptions. The demo can still work locally even if the backend is minimal.

Endpoint	Purpose	Response example	Priority
GET /health	Quick demo check that server runs	status ok	Must-have
GET /facts	Return 3 short ocean-impact facts for end section	JSON array of facts	Must-have
GET /nodes	Return node metadata, titles, and descriptions	JSON object by node id	Nice to have
POST /session	Optional lightweight logging of completed run	JSON summary	Stretch only

```
backend/
main.py
routers/
  facts.py
  nodes.py
data/
  facts.json
  nodes.json
requirements.txt
```

Do not overbuild the backend

- No auth, no user accounts, no relational database, no admin tools.
- If time gets tight, the site can hardcode the content and keep FastAPI only for a simple health route.

7. Data, Copy, and Content Plan

Keep the copy short and presentation-ready. This project wins on clarity, not word count.

Location	Exact content role	Length target	Notes
Hero subtitle	Sets fantasy immediately	1 sentence	A rogue shark versus a machine-controlled ocean
Mission copy	Explains player goal	2 to 3 lines	No heavy lore
Objective labels	Makes node purpose obvious	2 to 4 words each	Destroy, Cut, Disable
Final message	Converts experience into takeaway	2 to 3 lines	Oceans are living systems, not harvest machines
Fact cards	Adds real-world context	1 line per card	Three cards maximum

8. Integration Plan and MVP Build Order

Build stage	React work	Python work	Definition of done
Stage 1	Scaffold app, sections, layout	Start FastAPI app with health route	End-to-end page flow exists
Stage 2	Build playable reef scene shell	Add facts endpoint or static JSON	Scene renders and accepts input
Stage 3	Add 3 node interactions and progress state	Serve facts/nodes if desired	Full loop works technically
Stage 4	Polish transformation, visuals, and ending	Finalize tiny backend or leave minimal	Demo feels intentional and stable
Stage 5	Bug fix and rehearse	Verify local run commands	Team can present without confusion

9. Cut List and Final Recommendation

- **Never cut:** landing hook, playable scene, 3 nodes, restoration meter, final transformation, impact message.
- **Cut first:** sound effects, extra fish schools, advanced particles, session logging.
- **Cut second:** drag slicing if buggy; convert every node to simple click/hold interaction.
- **Avoid entirely:** multiplayer, login, large backend, heavy physics, 3D browser rendering unless already proven.

Best 24-hour version

- A React single-page site with one interactive reef scene and a tiny FastAPI backend is the highest-probability ship.
- The website should demo cleanly on localhost with one command for frontend and one command for backend.

10. Run Commands

Frontend

```
npm install  
npm run dev
```

Backend

```
pip install -r requirements.txt  
uvicorn main:app --reload
```

Local demo flow

- 1) Start FastAPI
- 2) Start React app
- 3) Open browser to local Vite URL
- 4) Play through 3 nodes
- 5) End on final message section